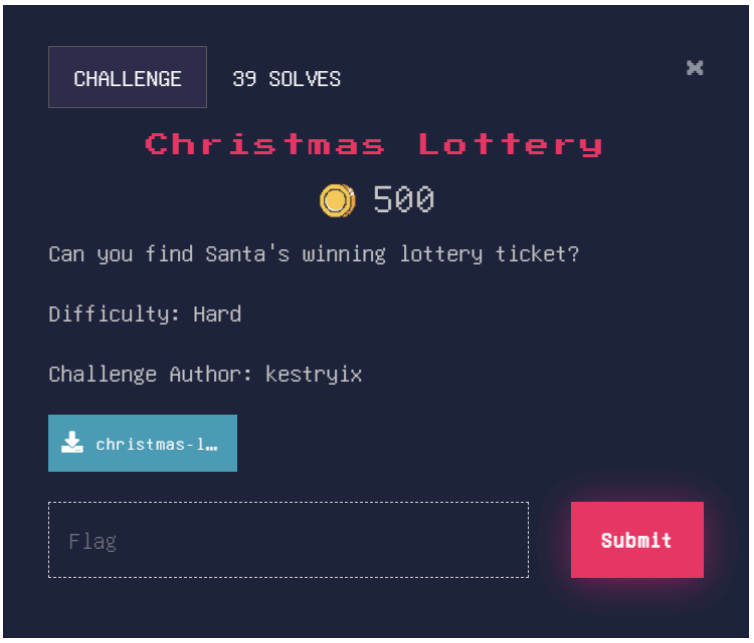
Christmas Lottery (REV - 500 pts)

Challenge Author: kestryix, Difficulty: Hard, Solve: 39 (As of 23/12/25)

Description:
Can you find Santa's winning lottery ticket?
Artefact: christmas-lottery.apk



Initial Analysis

Given an Android APK file, I began by performing static analysis to inspect the application's code and resources without executing it. This approach helps to identify hardcoded secrets, encryption logic, and flag construction mechanisms while avoiding potential anti-emulation or runtime protections.

Static Analysis

After decompiling the APK using jadx-gui, I focused on `MainActivity.java` (in com.christmas.lottery), as it contains the core application logic related to ticket generation and validation.

Key Observations

From the static analysis, several important methods were identified:

- calculateChecksum(String str)
- generateTicketCode()
- requestSeedFromBackend(String str)
- validateTicketCodeWithBackend(String str)

This indicates that ticket codes are generated locally using a deterministic algorithm, while validation is performed via the backend service.

Ticket Code Generation Logic

Seed-Based Generation

```
private String generateTicketCode() {
    StringBuilder sb = new StringBuilder();
    for (int i3 = 0; i3 < 3; i3++) {
        sb.append("ABCDEFGHJKLMNPQRSTUVWXYZ".charAt((int) ((this.secretSeed >> (i3 * 3)) % 24)));
    }
    sb.append("-");
    for (int i4 = 0; i4 < 4; i4++) {
        sb.append("0123456789".charAt((int) ((this.secretSeed >> ((i4 * 4) + 9)) % 10)));
    }
    sb.append("-");
    for (int i5 = 0; i5 < 2; i5++) {
        sb.append("ABCDEFGHJKLMNPQRSTUVWXYZ".charAt((int) ((this.secretSeed >> ((i5 * 5) + 25)) % 24)));
    }
    sb.append("0123456789".charAt(calculateChecksum(sb.toString().replace("-", "")) % 10));
    return sb.toString();
}
```

The generateTicketCode() method constructs ticket codes using a variable named secretSeed. The ticket format follows the structure AAA-1234-BB5, where each segment is derived from bitwise operations on the seed value.

This confirms that:

- Ticket generation is fully deterministic
- Anyone with the same secretSeed can recreate a valid ticket

Since ticket generation is fully deterministic, obtaining the secretSeed would allow reconstruction of a valid winning ticket. The next step was therefore to identify how the application retrieves this seed.

Backend Interaction

```
/* JADX INFO: Access modifiers changed from: private */
public void requestSeedFromBackend(String str) {
    this.executorService.execute(new RunnableC0314k(8, this, str));
}
```

Through the requestSeedFromBackend(String str) function, I discovered that the application retrieves secretSeed from a backend server. The backend URL is stored in the app resources. [this.backendUrl = getString(AbstractC0460d.backend_url);]

The app uses Firebase Anonymous Authentication

```
/* JADX INFO: Access modifiers changed from: private */
public void signInAnonymouslyAndInitSeed() {
    Task<Object> taskZza;
    this.mAuth.a();
    FirebaseAuth firebaseAuth = this.mAuth;
    AbstractC0365l abstractC0365l = firebaseAuth.f2530f;
    if (abstractC0365l == null || !abstractC0365l.c()) {
        taskZza = firebaseAuth.f2529e.zza(firebaseAuth.f2525a, new C0359f(firebaseAuth), firebaseAuth.f2533i);
    } else {
        C0384c c0384c = (C0384c) firebaseAuth.f2530f;
        c0384c.f3844m = false;
        taskZza = Tasks.forResult(new F(c0384c));
    }
    taskZza.addOnCompleteListener(this, new q(this, 15));
}
```

To find the backend URL, I looked into the res/values/strings.xml and found the backend configuration:

- URL: https://us-central1-christmas-lottery-b838b.cloudfunctions.net
- API Key: AIzaSyCci4KfQm1vcGIrBgdVM94hkbSMYmDiGi4 (Firebase API Key)

The networking logic was obfuscated in i/RunnableC0314k.java. By analysing Case 8 in that file, I discovered the API endpoint details

```
case 8:
    try {
        ArrayList arrayList = new ArrayList();
        ArrayList arrayList2 = new ArrayList();
        String str = (String) this.f3545b;
        kl.j.o(str, "value");
        char[] cArr = Wl.u.f1049k;
        arrayList.add(Wl.n.b("idToken", 0, 0, " \"':;<=>@[]^`{}|/\\?#&!$(),~", false, false, true, false, null, 91));
        arrayList2.add(Wl.n.b(str, 0, 0, " \"':;<=>@[]^`{}|/\\?#&!$(),~", false, false, true, false, null, 91));
        Wl.p pVar = new Wl.p(arrayList, arrayList2);
        Wl.z zVar = new Wl.z();
        zVar.d(((MainActivity) obj).backendUrl + "/init");
        zVar.c("POST", pVar);
        C0339x c0339xA = zVar.a();
        Wl.x xVar = ((MainActivity) obj).httpClient;
        xVar.getClass();
        bC = new a2.i(xVar, c0339xA, false).c();
        try {
            ((MainActivity) obj).runOnUiThread(new RunnableC0314k(6, this, new JSONObject(bC.f930j.c())));
            bC.close();
            return;
        } finally {
        }
    } catch (Exception e4) {
        ((MainActivity) obj).runOnUiThread(new RunnableC0314k(7, this, e4));
        return;
    }
```

- Endpoint: /init
- Method: POST
- Payload: It requires a Firebase idToken sent as Form Data (not JSON)

The Solution

There are two methods to solve this challenge. The first method requires an emulator to run, and the alternative method makes use of a Python script to emulate the application behavior.

Method 1: Using Frida

Requirements:

- Android Emulator (e.g, Genymotion, Android Studio) or Rooted Device
- frida-server
- The app installed and running

Setup

Download Genymotion, create a new device, and start the emulator. In this case, a Google Nexus 6 running Android 10 was used.

Type	Name	Android API	Resolution	Density	Size on disk	Source	Status	Actions
	Google Nexus 6	10.0 - API 29	1440 × 2560	560	1,011.58 MiB	Genymotion	On	

ADB Connect

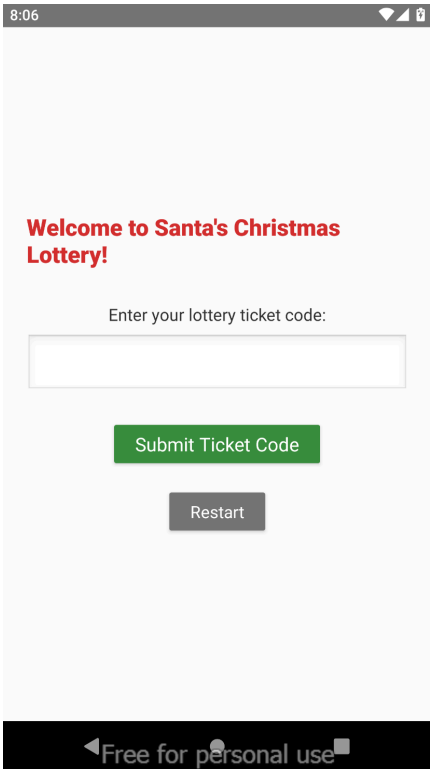
Connect to the IP address of the running emulator

```
┌──(kali㉿LAPTOP-D23LFTIU)-[~]
└─$ adb connect 192.168.56.101:5555
connected to 192.168.56.101:5555

┌──(kali㉿LAPTOP-D23LFTIU)-[~]
└─$ adb devices
List of devices attached
192.168.56.101:5555     device
```

Install the application in the emulator

Copy and paste or drag and drop the APK file into the emulator to install it. (You can use adb push too)



Start the Frida-server

adb push the frida server into the emulator and start the server

```
┌──(kali㉿LAPTOP-D23LFTIU)-[~/Hex]
└─$ adb push frida-server /data/local/tmp/
frida-server: 1 file pushed, 0 skipped. 20.3 MB/s (53079128 bytes in 2.498s)

┌──(kali㉿LAPTOP-D23LFTIU)-[~/Hex]
└─$ adb shell "chmod 755 /data/local/tmp/frida-server"

┌──(kali㉿LAPTOP-D23LFTIU)-[~/Hex]
└─$ adb shell "/data/local/tmp/frida-server &"
```

Verify that the Frida server has started

```
┌──(ctf-env)(kali㉿LAPTOP-D23LFTIU)-[~]
└─$ frida-ps -U
 PID  Name
----  ----------------------------------------------
2876  Christmas Lottery
2585  Email
2640  Gallery
2379  Messaging
2439  Phone
1354  Settings
2749  Superuser
 550  adbd
```

Frida Script

Initial attempts to directly invoke the ticket generation logic using Frida were unreliable due to asynchronous seed initialization and obfuscated control flow. As a result, a data-flow–based approach was adopted, focusing on intercepting sensitive values at runtime rather than forcing execution of private methods.

Identifying a Reliable Hook Point

From the static analysis and backend investigation, it was clear that the `/init` endpoint returns a JSON object containing the seed value. This response is parsed within the application using the standard Android JSON library:

`org.json.JSONObject.getLong("seed")`

This method provides a stable and unobfuscated hook point that is independent of application-specific control flow. By intercepting this function, the secret seed can be captured at the exact moment it is parsed from the backend response.

Frida Hook Implementation

Using Frida, I hooked the `getLong(String key)` method of org.json.JSONObject and monitored for access to the “seed” key. When detected, the seed value was extracted directly from memory.

```
const JSONObject = Java.use("org.json.JSONObject");

JSONObject.getLong.overload("java.lang.String").implementation = function (key) {
    const value = this.getLong(key);

    if (key === "seed" && value !== 0) {
        console.log("\n[!!!] secretSeed captured from JSON:", value.toString());

        const ticket = generateTicket(value);
        console.log("\n    [$$$] WINNING TICKET:", ticket, "\n");
    }

    return value;
};
```

This approach avoids

- Lifecycle race conditions
- Garbage collection issues
- Reliance on obfuscated class names
- Premature access to uninitialised variables

Ticket Generation and Flag Retrieval

Once the seed was captured at runtime, the previously reversed ticket generation algorithm was reimplemented within the Frida script to generate a valid lottery ticket.

```
Java.perform(function () {

    // --- Ticket logic (same as app) ---
    function calculateChecksum(str) {
        let sum = 0;
        for (let i = 0; i < str.length; i++) {
            const c = str.charAt(i);
            if (c >= '0' && c <= '9') sum += (c.charCodeAt(0) - 48);
            else sum += (c.charCodeAt(0) - 64);
        }
        return sum % 10;
    }

    function generateTicket(seed) {
        const alphabet = "ABCDEFGHJKLMNPQRSTUVWXYZ";
        const digits = "0123456789";
        let sb = "";

        for (let i = 0; i < 3; i++)
            sb += alphabet[(seed >> (i * 3)) % 24];
        sb += "-";

        for (let i = 0; i < 4; i++)
            sb += digits[(seed >> ((i * 4) + 9)) % 10];
        sb += "-";

        for (let i = 0; i < 2; i++)
            sb += alphabet[(seed >> ((i * 5) + 25)) % 24];

        const clean = sb.replace(/-/g, "");
        sb += digits[calculateChecksum(clean)];

        return sb;
    }
```

Execution

Save the script and run with Frida

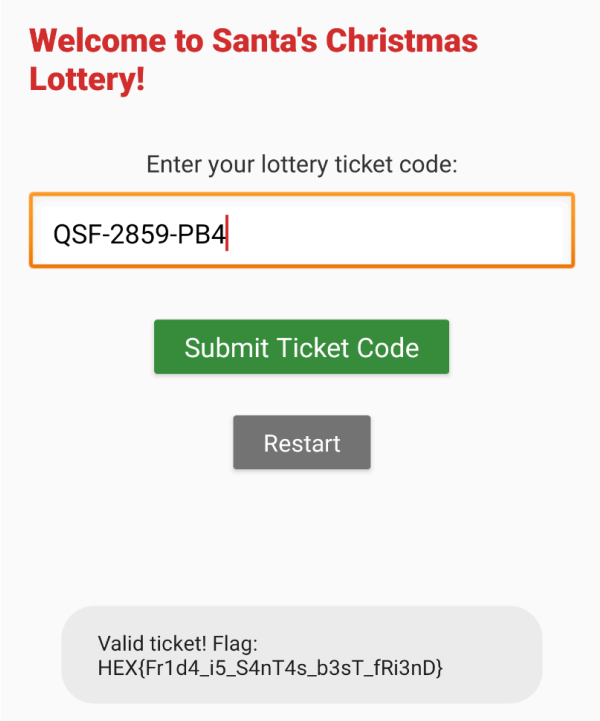
`frida -U -f com.christmas.lottery -l fridaSolve.js`

Result

The Frida script successfully intercepted the backend response at runtime and extracted the `secretSeed` value at the moment it was parsed. Using the recovered seed, a valid lottery ticket was generated and submitted to the application, which successfully returned the flag.

```
┌──(ctf-env)(kali㉿LAPTOP-D23LFTIU)-[~/Hex]
└─$ frida -U -f com.christmas.lottery -l fridaSolve.js
     ____
    / _  |   Frida 17.5.2 - A world-class dynamic instrumentation toolkit
   | (_| |
    > _  |   Commands:
   /_/ |_|       help      -> Displays the help system
   . . . .       object?   -> Display information about 'object'
   . . . .       exit/quit -> Exit
   . . . .
   . . . .   More info at https://frida.re/docs/home/
   . . . .
   . . . .   Connected to Nexus 6 (id=192.168.56.101:5555)
Spawning `com.christmas.lottery`...
[*] Frida attached - hooking JSONObject.getLong("seed")
Spawned `com.christmas.lottery`. Resuming main thread!
[Nexus 6::com.christmas.lottery ]->
[!!!] secretSeed captured from JSON: 1256393030

    [$$$] WINNING TICKET: QSF-2859-PB4
```



Alternative Method: Using a Python Script

As an alternative, the challenge can also be solved without dynamic instrumentation by replicating the application's backend interaction and ticket generation logic externally.

1. Authenticate with Firebase to get a valid `idToken`.
2. POST the token to `/init` to get the secretSeed.
3. Reimplement the Java `generateTicketCode` logic in Python.
4. Submit the resulting ticket to the `/validate` endpoint.

Configuration

The configuration values are from what we have found earlier during static analysis.

```python
import requests
import json

# --- CONFIGURATION ---
API_KEY = "AIzaSyCci4KfQm1vcGIrBgdVM94hkbSMYmDiGi4"
BASE_URL = "https://us-central1-christmas-lottery-b838b.cloudfunctions.net"
```

Ticket Code Generation Logic (In Python)

This section replicates the application's ticket generation logic in Python.

```python
# --- TICKET GENERATION LOGIC ---
def calculate_checksum(s):
    numeric_value = 0
    for char in s:
        if char.isdigit():
            val = int(char)
        else:
            val = ord(char) - 64
        numeric_value += val
    return numeric_value % 10

def generate_winning_ticket(secret_seed):
    alphabet = "ABCDEFGHJKLMNPQRSTUVWXYZ"
    digits = "0123456789"
    sb = ""

    # Part 1
    for i in range(3):
        sb += alphabet[(secret_seed >> (i * 3)) % 24]
    sb += "-"

    # Part 2
    for i in range(4):
        sb += digits[(secret_seed >> ((i * 4) + 9)) % 10]
    sb += "-"

    # Part 3
    for i in range(2):
        sb += alphabet[(secret_seed >> ((i * 5) + 25)) % 24]

    # Checksum
    clean_str = sb.replace("-", "")
    checksum = calculate_checksum(clean_str) % 10
    sb += digits[checksum]

    return sb
```

Connect and authenticate to the backend

This function connects to the Firebase backend with the API key and retrieves the `idToken` required for the exploit.

```python
def get_firebase_token():
    print("[1] Authenticating with Firebase...")
    url = f"https://identitytoolkit.googleapis.com/v1/accounts:signUp?key={API_KEY}"
    r = requests.post(url, json={"returnSecureToken": True})
    if r.status_code == 200:
        print("    [+] Got ID Token!")
        return r.json()["idToken"]
    else:
        print(f"    [-] Auth Failed: {r.text}")
        exit(1)
```

Get secretSeed

Retrieves secretSeed from `/init`. Note the use of `data=` to send the payload as Form Data instead of JSON so that the server accepts the request.

```python
def get_secret_seed(token):
    print("[2] Fetching Secret Seed from /init...")
    url = f"{BASE_URL}/init"

    # Send as 'data' (Form-Encoded), NOT 'json'
    payload = {"idToken": token}

    r = requests.post(url, data=payload)

    if r.status_code == 200:
        data = r.json()
        if "seed" in data:
            seed = int(data["seed"])
            print(f"    [+] Seed received: {seed}")
            return seed

    print(f"    [-] Failed to get seed. Status: {r.status_code}")
    print(f"    [-] Response: {r.text}")
    exit(1)
```

Submit the Ticket

Submit the calculated ticket code to the `/validate` endpoint to retrieve the final flag.

```python
def get_flag(token, ticket_code):
    print(f"[3] Submitting Ticket {ticket_code} to /validate...")
    url = f"{BASE_URL}/validate"

    # From Case 10, it sends ticketCode and idToken
    payload = {
        "idToken": token,
        "ticketCode": ticket_code
    }

    r = requests.post(url, data=payload)

    if r.status_code == 200:
        data = r.json()
        if "flag" in data:
            return data["flag"]

    print(f"    [-] Failed to validate. Status: {r.status_code}")
    print(f"    [-] Response: {r.text}")
    return "???"
```

Main

```python
# --- MAIN ---
if __name__ == "__main__":
    # Step 1: Login
    token = get_firebase_token()

    # Step 2: Get Seed
    seed = get_secret_seed(token)

    # Step 3: Calculate Ticket
    ticket = generate_winning_ticket(seed)
    print(f"    [+] Generated Ticket: {ticket}")

    # Step 4: Profit
    flag = get_flag(token, ticket)

    print("\n" + "="*40)
    print(f"FLAG: {flag}")
    print("="*40)
```

Result

Executing the script successfully returned the flag:

```
┌──(kali㉿LAPTOP-D23LFTIU)-[~/Hex]
└─$ python3 solve_lottery.py
[1] Authenticating with Firebase...
    [+] Got ID Token!
[2] Fetching Secret Seed from /init...
    [+] Seed received: 532325810
    [+] Generated Ticket: CGG-8113-RA9
[3] Submitting Ticket CGG-8113-RA9 to /validate...

========================================
FLAG: HEX{Fr1d4_i5_S4nT4s_b3sT_fRi3nD}
========================================
```

Flag: **HEX{Fr1d4_i5_S4nT4s_b3sT_fRi3nD}**

Conclusion

This challenge required a combination of static and dynamic analysis to understand the application's behavior and successfully recover the winning lottery ticket. Static analysis was used to reverse the ticket generation logic and identify how the seed was obtained, while dynamic analysis was applied to observe the application's runtime behavior and extract the required value. Together, these techniques allowed the winning ticket to be reconstructed and the flag to be retrieved.

Writeup by: PandaSploit
Date: 23/12/2025